

Domain Granted Powers — Implementation Plan

Date: 2026-05-21

Project: /home/ubuntu/dnd35prototype

Scope: All 22 PHB cleric domain granted powers (mechanical implementation)

Executive Summary

Current State

The codebase has a solid **domain infrastructure** but **zero mechanical implementations** of domain granted powers:

-  `DomainData` class stores domain name, granted power description, and spell lists
-  `DomainDatabase` registers all 22 domains with text descriptions of granted powers
-  `CharacterStats.ChosenDomains` (List) tracks selected domains per character
-  `DeityData.Domains` lists available domains per deity; `DeityData.FavoredWeapon` stores favored weapon
-  `DomainSelectionUI` allows picking 2 domains during character creation
-  Domain spells slot system works (extra spell slot per level)
-  **No granted powers are mechanically applied** — they're text-only descriptions
-  No `GetCasterLevel()` hook for domain CL bonuses
-  No domain feat granting (War domain Weapon Focus)
-  No elemental turning system (Air/Earth/Fire/Water)
-  No activated ability system (Death Touch, Destruction Smite, etc.)
-  No domain class skill additions (Trickery, Animal, Knowledge, Plant, Travel)

What Exists That We Can Leverage

System	Location	Reuse For
Turn Undead	<code>TurnUndeadSystem.cs</code> (~993 lines)	Elemental turning (Air/Earth/Fire/Water/Plant), Greater Turning (Sun)
Template Smite	<code>TemplateSmiteSystem.cs</code> (~247 lines)	Destruction domain smite
Feat system	<code>FeatDefinitions.cs</code> , <code>CharacterStats.Feats</code>	War domain Weapon Focus grant
Freedom of Movement	<code>CharacterStats.FreedomOfMovementActive</code>	Travel domain movement freedom
Class skills	<code>ClassSkillDefinitions.GetClassSkills()</code>	Trickery/Animal/Knowledge/Plant/Travel class skills
Difficult terrain	<code>MovementService.IsDifficultTerrain()</code>	Travel domain immunity
<code>GetCasterLevel()</code>	<code>CharacterStats.cs:1324</code>	Domain CL bonus hook
Deity favored weapon	<code>DeityData.FavoredWeapon</code>	War domain weapon proficiency + focus

Domain Power Status Matrix

#	Domain	Power Type	Complexity	Status	Tier
1	Good	+1 CL (good spells)	Low	✗ Missing	1
2	Evil	+1 CL (evil spells)	Low	✗ Missing	1
3	Law	+1 CL (law spells)	Low	✗ Missing	1
4	Chaos	+1 CL (chaos spells)	Low	✗ Missing	1
5	Healing	+1 CL (healing spells)	Low	✗ Missing	1
6	Knowledge	+1 CL (divination) + class skills	Low	✗ Missing	1
7	Trickery	Add Bluff/Disguise/Hide as class skills	Low	✗ Missing	2
8	Animal	Speak with Animals 1/day + class skill	Low	✗ Missing	2
9	Plant	Rebuke/command plants + class skill	Med	✗ Missing	3
10	Travel	Freedom of movement rounds + class skill	Med	✗ Missing	2
11	War	Weapon Focus + proficiency (deity weapon)	Med	✗ Missing	2
12	Protection	Protective ward (resistance bonus to next save)	Med	✗ Missing	3

#	Domain	Power Type	Complexity	Status	Tier
13	Strength	+CL enhancement to STR, 1 round, 1/day	Med	✗ Missing	3
14	Sun	Greater Turning (destroy undead) 1/day	Med	✗ Missing	3
15	Destruction	Smite (+4 attack, +CL damage) 1/day	Med	✗ Missing	3
16	Death	Death Touch (1d6/CL vs HP) 1/day	Med	✗ Missing	3
17	Luck	Reroll any roll 1/day	High	✗ Missing	4
18	Magic	Use spell completion items as half-CL wizard	Low	✗ Missing	4
19	Air	Turn earth / rebuke air creatures	High	✗ Missing	5
20	Earth	Turn air / rebuke earth creatures	High	✗ Missing	5
21	Fire	Turn water / rebuke fire creatures	High	✗ Missing	5
22	Water	Turn fire / rebuke water creatures	High	✗ Missing	5

Tier-Based Implementation Plan

Tier 1: Caster Level Bonuses (6 domains — Quick Wins)

Domains: Good, Evil, Law, Chaos, Healing, Knowledge

Effort: Small — single hook in `GetCasterLevel()` + spell descriptor tagging

Dependencies: Need a spell descriptor/tag system on `SpellData`

Technical Approach

Problem: `SpellData` has no `Descriptor` field. D&D 3.5e spell descriptors include: Good, Evil, Law, Chaos, Fear, Mind-Affecting, etc. We need to tag spells so domain CL bonuses can apply.

Step 1: Add Descriptors to SpellData (`Assets/Scripts/Magic/SpellData.cs`)

```
// Add after School field (~line 47)
/// <summary>Spell descriptors (e.g., "Good", "Evil", "Law", "Chaos", "Healing",
"Divination").</summary>
public List<string> Descriptors = new List<string>();

public bool HasDescriptor(string descriptor)
{
    if (Descriptors == null || string.IsNullOrEmpty(descriptor)) return false;
    for (int i = 0; i < Descriptors.Count; i++)
        if (string.Equals(Descriptors[i], descriptor, StringComparison.OrdinalIgnoreCase))
            return true;
    return false;
}
```

Step 2: Tag existing spells with descriptors (in each `SpellDatabase_*.cs` file)

- Protection from Evil → `Descriptors = { "Good" }`
- Protection from Good → `Descriptors = { "Evil" }`
- Protection from Law → `Descriptors = { "Chaos" }`
- Protection from Chaos → `Descriptors = { "Law" }`
- Aid → `Descriptors = { "Good" }`
- All Cure spells → `Descriptors = { "Healing" }`
- All Inflict spells → `Descriptors = { "Evil" }` (arguably)
- Holy Smite → `Descriptors = { "Good" }`
- Chaos Hammer → `Descriptors = { "Chaos" }`
- Order's Wrath → `Descriptors = { "Law" }`
- Unholy Blight → `Descriptors = { "Evil" }`
- Detect spells, Identify, Locate Object → School already "Divination" (use School for Knowledge domain)

Step 3: Create `DomainPowerService` (new file: `Assets/Scripts/Character/DomainPowerService.cs`)

```

public static class DomainPowerService
{
    /// <summary>
    /// Get domain-based caster level bonus for a specific spell.
    /// Called from GetEffectiveCasterLevel() during spell resolution.
    /// </summary>
    public static int GetDomainCasterLevelBonus(CharacterStats stats, SpellData spell)
    {
        if (stats == null || spell == null || stats.ChosenDomains == null)
            return 0;

        int bonus = 0;
        foreach (string domain in stats.ChosenDomains)
        {
            switch (domain)
            {
                case "Good": if (spell.HasDescriptor("Good")) bonus++; break;
                case "Evil": if (spell.HasDescriptor("Evil")) bonus++; break;
                case "Law": if (spell.HasDescriptor("Law")) bonus++; break;
                case "Chaos": if (spell.HasDescriptor("Chaos")) bonus++; break;
                case "Healing": if (spell.HasDescriptor("Healing")) bonus++; break;
                case "Knowledge":
                    if (string.Equals(spell.School, "Divination", StringComparison.OrdinalIgnoreCase))
                        bonus++;
                    break;
            }
        }
        return bonus;
    }
}

```

Step 4: Hook into spell resolution (Assets/Scripts/Core/GameManager.SpellCasting.cs)

Find where `casterLevel` is calculated during spell resolution and add:

```

int domainCLBonus = DomainPowerService.GetDomainCasterLevelBonus(caster.Stats, spell);
casterLevel += domainCLBonus;

```

Files to modify:

- Assets/Scripts/Magic/SpellData.cs — Add `Descriptors` field + `HasDescriptor()`
- Assets/Scripts/Magic/Spells/Databases/SpellDatabase_*.cs — Tag spells with descriptors
- **NEW** Assets/Scripts/Character/DomainPowerService.cs — Central domain power logic
- Assets/Scripts/Core/GameManager.SpellCasting.cs — Hook CL bonus into spell resolution

Tier 2: Passive Grants (4 domains — Class Skills, Feats, Movement)

Domains: Trickery, Animal, War, Travel

Effort: Small-Medium — applied at character creation/finalization time

2A: Class Skill Additions (Trickery, Animal, Knowledge, Plant, Travel)

Technical Approach:

The class skill system uses `ClassSkillDefinitions.GetClassSkills(className)` which returns `ICharacterClass.ClassSkills`. Domain class skills need to be injected **after** domain selection.

Option A (Recommended): Modify `CharacterStats.IsClassSkill()` check

Find where class skills are checked (likely in `CharacterStats` skill calculation or character sheet) and add a domain overlay:

```
// In DomainPowerService.cs
public static HashSet<string> GetDomainClassSkills(List<string> chosenDomains)
{
    var skills = new HashSet<string>();
    if (chosenDomains == null) return skills;
    foreach (string domain in chosenDomains)
    {
        switch (domain)
        {
            case "Trickery": skills.Add("Bluff"); skills.Add("Hide"); break;
            // Note: "Disguise" not in prototype skill list – skip or add
            case "Animal": skills.Add("Knowledge (Nature)");
            break; // May not be in skill list
            case "Knowledge": /* All Knowledge skills – add if in list */ break;
            case "Plant": skills.Add("Knowledge (Nature)"); break;
            case "Travel": skills.Add("Survival"); break; // May not be in list
        }
    }
    return skills;
}
```

Caveat: The prototype skill list (`ClassSkillDefinitions.AllSkills`) is limited — it doesn't include Knowledge (Nature), Knowledge (Religion) as separate entries, Disguise, or Survival. Domain class skills may have limited practical effect in the current prototype. This can be noted and implemented when the skill list expands.

Current prototype skills: Appraise, Balance, Bluff, Climb, Diplomacy, Disable Device, Gather Information, Hide, Intimidate, Jump, Listen, Move Silently, Open Lock, Search, Sleight of Hand, Spot, Swim, Tumble, Use Magic Device

Actionable now: Trickery adds Bluff + Hide (both in list!). Others depend on skill list expansion.

Files to modify:

- `Assets/Scripts/Character/DomainPowerService.cs` — `GetDomainClassSkills()`
- `Assets/Scripts/Character/CharacterStats.cs` — Hook domain skills into skill calculation (find `IsClassSkill` or equivalent)

2B: War Domain — Weapon Focus + Proficiency

Technical Approach:

At character finalization (when domains are applied to `CharacterStats`), check if “War” is in `ChosenDomains` :

1. Look up `stats.Deity.FavoredWeapon` (e.g., “Longsword” for Heironeous)
2. Grant martial weapon proficiency for that weapon (if not already proficient)
3. Grant “Weapon Focus” feat via `stats.Feats.Add("Weapon Focus")`
4. Set `stats.WeaponFocusWeapon = favoredWeapon` (or equivalent field)

Integration point: `Assets/Scripts/Core/GameManager.cs:~1821` where `stats.ChosenDomains` is set from creation data. Add a call to `DomainPowerService.ApplyDomainGrantedPowers(stats)` after domain assignment.

Files to modify:

- Assets/Scripts/Character/DomainPowerService.cs — ApplyWarDomainFeats(stats)
- Assets/Scripts/Core/GameManager.cs — Call ApplyDomainGrantedPowers() after domain assignment
- Assets/Scripts/Character/CharacterStats.cs — May need WeaponFocusWeapon field if not already present

2C: Travel Domain — Movement Freedom**Technical Approach:**

Travel domain grants 1 round/CL per day of freedom from movement-impairing effects. The existing FreedomOfMovementActive bool and FreedomOfMovementRoundsRemaining fields on CharacterStats can be leveraged.

Add to DomainPowerService :

```
public static void InitTravelDomainFreedom(CharacterStats stats)
{
    if (stats.ChosenDomains.Contains("Travel"))
    {
        stats.TravelDomainRoundsPerDay = stats.GetCasterLevel();
        stats.TravelDomainRoundsUsed = 0;
    }
}
```

New fields needed on CharacterStats :

- int TravelDomainRoundsPerDay
- int TravelDomainRoundsUsed
- bool TravelDomainActive (toggleable via UI button)

Integration: When active, sets FreedomOfMovementActive = true and decrements rounds. Expose as an action button.

Tier 3: Activated Abilities (6 domains — 1/day powers)

Domains: Strength, Protection, Sun, Destruction, Death, Plant

Effort: Medium — each needs UI button, usage tracking, and effect resolution

Common Infrastructure Needed**1/day ability tracking pattern:**

```
// Add to CharacterStats.cs
public bool DestructionSmiteUsed;
public bool DeathTouchUsed;
public bool StrengthFeatUsed;
public bool ProtectiveWardUsed;
public bool GreaterTurningUsed;
```

These reset at long rest / combat start (find the reset logic near TurnUndeadAttemptsUsedToday reset).

UI integration: Each activated power needs an action button. Follow the pattern in ActionButtonPanel.ComputeActionButtonStates(). Add domain power buttons conditionally when the cleric has the relevant domain.

3A: Destruction Domain — Smite

Pattern: Nearly identical to `TemplateSmiteSystem.cs`.

- +4 attack bonus, +CL damage on one melee attack
- 1/day, standard action
- Can target anyone (not alignment-restricted like paladin smite)

Approach: Add a new smite variant to `TemplateSmiteSystem` or create `DomainSmiteSystem.cs`. Key difference: no alignment targeting restriction.

Files:

- **NEW** `Assets/Scripts/CombatSystems/DestructionSmiteSystem.cs` (or extend `TemplateSmiteSystem`)
- `Assets/Scripts/UI/Panels/ActionButtonPanel.cs` — Add smite button for Destruction domain
- `Assets/Scripts/Character/CharacterStats.cs` — `DestructionSmiteUsed` field

3B: Death Domain — Death Touch

Mechanics: Melee touch attack → roll 1d6 per CL → if total $\geq$ target's current HP, target dies (no save).

Approach: Create `DeathTouchSystem.cs` as a `GameManager` partial class:

1. Touch attack roll (BAB + STR mod vs touch AC)
2. Roll `CL × d6`
3. Compare to target's current HP
4. If $\geq$, instant death; otherwise no effect

Files:

- **NEW** `Assets/Scripts/Core/GameManager_DeathTouch.cs`
- `Assets/Scripts/UI/Panels/ActionButtonPanel.cs` — Death Touch button
- `Assets/Scripts/Character/CharacterStats.cs` — `DeathTouchUsed` field

3C: Strength Domain — Feat of Strength

Mechanics: Enhancement bonus to STR = cleric level, lasts 1 round, 1/day.

Approach: Apply temporary STR enhancement bonus:

```
stats.StrengthEnhancementBonus = Mathf.Max(stats.StrengthEnhancementBonus, clericLevel);
// Set to expire after 1 round
```

Need to track and clear after 1 round. Could use `StatusEffectManager.AddEffect()` with a 1-round duration.

Files:

- `Assets/Scripts/Character/DomainPowerService.cs` — `ActivateFeatOfStrength()`
- `Assets/Scripts/Character/CharacterStats.cs` — Ensure STR enhancement bonus field exists
- `Assets/Scripts/UI/Panels/ActionButtonPanel.cs` — Button

3D: Protection Domain — Protective Ward

Mechanics: Touch ally → grant resistance bonus = CL to their next saving throw. Discharged on first save.

Approach: Add fields to `CharacterStats`:

```
public int ProtectiveWardBonus; // Resistance bonus to next save
```

Hook into saving throw calculation to add `ProtectiveWardBonus`, then clear it after use.

Files:

- `Assets/Scripts/Character/CharacterStats.cs` — `ProtectiveWardBonus` field, hook into saves
- `Assets/Scripts/Character/DomainPowerService.cs` — `ActivateProtectiveWard()`
- Save calculation code (find `WillSave`, `FortSave`, `ReflexSave` properties)

3E: Sun Domain — Greater Turning

Mechanics: 1/day, replace a normal turn undead attempt. Turned undead are **destroyed** instead of fleeing. Effective turning level = CL + 4.

Approach: Modify `TurnUndeadSystem.ExecuteTurnUndead()`:

1. Add a `isGreaterTurning` flag
2. When resolving, use `effectiveTurnLevel = CL + 4`
3. Turned targets are destroyed (HP → 0, death) instead of applying Turned condition

Files:

- `Assets/Scripts/CombatSystems/TurnUndeadSystem.cs` — Add greater turning branch
- `Assets/Scripts/Character/CharacterStats.cs` — `GreaterTurningUsed` field
- `Assets/Scripts/UI/Panels/ActionButtonPanel.cs` — Greater Turning button (or modify Turn Undead button)

3F: Plant Domain — Rebuke/Command Plants

Mechanics: Rebuke or command plant creatures as an evil cleric rebukes undead. Uses same mechanics as turn undead but targets Plant creature type instead of Undead, and rebukes/commands instead of turns/destroys.

Approach: Generalize `TurnUndeadSystem` to support alternative creature types:

1. Add a `TurnTargetType` parameter (Undead, Plant, Elemental subtype)
2. The check/damage mechanics are identical
3. “Rebuke” = creature cowers for 10 rounds; “Command” = creature obeys (if HD ≤ cleric level)

This is the foundation for Tier 5 elemental turning as well.

Files:

- `Assets/Scripts/CombatSystems/TurnUndeadSystem.cs` — Generalize to support creature type parameter
- `Assets/Scripts/Character/CharacterStats.cs` — `PlantRebukeAttemptsUsed` (uses same 3+CHA pool or separate?)

Tier 4: Special Mechanics (2 domains)

Domains: Luck, Magic

Effort: Medium-High — unique systems

4A: Luck Domain — Reroll

Mechanics: 1/day, reroll any single roll before outcome is declared. Must take the reroll.

Challenge: This requires intercepting **any** d20 roll (attack, save, skill check, turn check, etc.) and offering a reroll option. This is architecturally complex because rolls happen in many places.

Approach Options:

Option A (Recommended): Post-roll prompt

After any d20 roll that affects the Luck domain cleric (or that they made), show a prompt:

“Use Luck Reroll? (1/day) — Current roll: X”

If accepted, reroll and use the new result.

Implementation:

1. Create a `LuckRerollService` that wraps roll resolution
2. Hook into key roll points:
 - Attack rolls (player attacks and saves vs player)
 - Saving throws (player saves)
 - Turn undead checks
3. UI: Modal prompt or automatic button that appears after a roll

This is the hardest domain power due to its cross-cutting nature. Consider implementing it as a “reroll prompt” that only appears for the cleric’s own rolls initially.

Files:

- **NEW** `Assets/Scripts/Character/LuckRerollService.cs`
- Multiple combat resolution files — hook points
- UI components for reroll prompt

4B: Magic Domain — Spell Completion Items

Mechanics: Use scrolls and wands as a wizard of half cleric level (min 1).

Challenge: The prototype’s magic item usage system needs to be checked. If wands/scrolls have a “required caster class” check, the Magic domain would bypass it for divine casters.

Approach: When a cleric with Magic domain attempts to use a scroll or wand:

1. Treat as if they were a wizard of level = $\max(1, \text{clericLevel} / 2)$
2. This may involve UMD (Use Magic Device) skill check bypass

Low priority — depends on how much the scroll/wand system is implemented. Likely a simple flag check.

Tier 5: Elemental Turning (4 domains)

Domains: Air, Earth, Fire, Water

Effort: High — requires generalizing the Turn Undead system

Technical Approach

Each elemental domain grants two abilities:

- **Turn** one element type (as good cleric turns undead)
- **Rebuke/Command** the opposite element type (as evil cleric rebukes undead)

Domain	Turn (destroy)	Rebuke (command)
Air	Earth creatures	Air creatures
Earth	Air creatures	Earth creatures
Fire	Water creatures	Fire creatures
Water	Fire creatures	Water creatures

Prerequisites:

1. Generalize `TurnUndeadSystem` to accept target creature type/subtype
2. Implement “Rebuke” variant (cower/command instead of flee/destroy)
3. Add separate daily use tracking (3 + CHA mod, same as turn undead)
4. Add UI buttons for each turning type

Step 1: Refactor `TurnUndeadSystem`

Extract the core turn/rebuke logic into a parameterized method:

```
private void ExecuteTurning(
    CharacterController cleric,
    string targetCreatureType,    // "Undead", "Plant", "Elemental"
    string targetSubtype,        // null, "Earth", "Air", "Fire", "Water"
    TurningMode mode,           // Turn, Rebuke, GreaterTurning
    int effectiveLevelBonus = 0)
```

Step 2: Creature subtype tagging

`CharacterStats.CreatureType` currently stores “Undead”, “Animal”, “Plant”, etc. Elemental creatures need subtypes: “Fire”, “Water”, “Earth”, “Air”. Options:

- Use `CreatureTags` list: `{"Elemental", "Fire"}`
- Add `CreatureSubtype` field

Step 3: Separate usage pools

Each elemental turning uses its own 3+CHA pool, separate from turn undead:

```
public int AirDomainTurnAttemptsUsed;
public int EarthDomainTurnAttemptsUsed;
public int FireDomainTurnAttemptsUsed;
public int WaterDomainTurnAttemptsUsed;
```

Files:

- `Assets/Scripts/CombatSystems/TurnUndeadSystem.cs` — Major refactor to generalize
 - `Assets/Scripts/Character/CharacterStats.cs` — Elemental turning pools, creature subtypes
 - `Assets/Scripts/UI/Panels/ActionButtonPanel.cs` — Elemental turn/rebuke buttons
 - Potentially **NEW** `Assets/Scripts/CombatSystems/ElementalTurningSystem.cs`
-

Implementation Dependencies

Tier 1 (CL Bonuses)

- ☐ SpellData.Descriptors field (prerequisite)
- ☐ DomainPowerService.cs (new file)
- ☐ GetCasterLevel() hook

Tier 2 (Passive Grants)

- ☐ DomainPowerService.ApplyDomainGrantedPowers() (called at char creation)
- ☐ War: DeityData.FavoredWeapon lookup
- ☐ Travel: New CharacterStats fields

Tier 3 (Activated Abilities)

- ☐ DomainPowerService.cs (ability activation methods)
- ☐ ActionButtonPanel.cs (UI buttons)
- ☐ CharacterStats.cs (usage tracking fields)
- ☐ Sun Greater Turning depends on Tier 5 refactor of TurnUndeadSystem (OR can be implemented **as** a simpler special **case** first)

Tier 4 (Special)

- ☐ Luck: Cross-cutting roll interception
- ☐ Magic: Depends on scroll/wand system maturity

Tier 5 (Elemental Turning)

- ☐ TurnUndeadSystem refactor (significant)
- ☐ Creature subtype system
- ☐ Plant domain (Tier 3F) **is** a simpler version of this

Recommended Implementation Order

Phase 1: Foundation + Quick Wins (Tiers 1-2)

~8-12 new/modified files, ~300-500 lines of new code

1. **Create** `DomainPowerService.cs` — Central service for all domain power logic
2. **Add** `SpellData.Descriptors` **field** — Tag spells with alignment/healing/divination descriptors
3. **Implement CL bonuses** (Good, Evil, Law, Chaos, Healing, Knowledge) — Hook into spell resolution
4. **Implement class skill additions** (Trickery: Bluff+Hide) — Hook into skill calculation
5. **Implement War domain** — Feat grant at character creation
6. **Call** `ApplyDomainGrantedPowers()` from GameManager character finalization

Phase 2: Activated Powers (Tier 3 minus Plant/Sun)

~4-6 new files, ~400-600 lines

1. **Destruction Smite** — Follow TemplateSmiteSystem pattern
2. **Death Touch** — New touch attack resolution
3. **Feat of Strength** — Temporary STR buff, 1 round
4. **Protective Ward** — Save bonus tracking + discharge

Phase 3: Turning Generalization (Tiers 3F, 3E, 5)

~200-400 lines of refactored code + new code

- 1. **Refactor TurnUndeadSystem** — Extract parameterized core
- 2. **Plant domain rebuke** — First non-undead turning
- 3. **Sun domain Greater Turning** — Destroy instead of flee
- 4. **Elemental turning** (Air, Earth, Fire, Water) — Full generalization

Phase 4: Special Mechanics (Tier 4)

~100-200 lines

- 1. **Magic domain** — Spell completion item bypass
- 2. **Luck domain reroll** — Cross-cutting roll interception

Phase 5: Travel Domain Full Implementation

~100-150 lines

- 1. **Travel domain freedom** — Round tracking, UI toggle, movement immunity

File-by-File Implementation Guidance

New Files to Create

File	Purpose
Assets/Scripts/Character/DomainPowerService.cs	Central service: CL bonuses, class skills, feat grants, ability activation
Assets/Scripts/Core/GameManager_DeathTouch.cs	Death domain touch attack resolution
Assets/Scripts/CombatSystems/DestructionSmiteSystem.cs	Destruction domain smite (or extend TemplateSmiteSystem)
Assets/Scripts/Character/LuckRerollService.cs	Luck domain reroll system

Files to Modify

File	Changes
Assets/Scripts/Magic/SpellData.cs	Add <code>Descriptors</code> list + <code>HasDescriptor()</code> method
Assets/Scripts/Magic/Spells/Databases/SpellDatabase_*.cs	Tag ~20-30 spells with descriptors
Assets/Scripts/Character/CharacterStats.cs	Add domain power usage fields, Travel domain fields, <code>ProtectiveWardBonus</code>
Assets/Scripts/Core/GameManager.cs	Call <code>DomainPowerService.ApplyDomainGrantedPowers()</code> at ~line 1822
Assets/Scripts/Core/GameManager.SpellCasting.cs	Hook domain CL bonus into spell resolution
Assets/Scripts/CombatSystems/TurnUndeadSystem.cs	Generalize for creature types, add greater turning
Assets/Scripts/UI/Panels/ActionButtonPanel.cs	Add domain power action buttons
Assets/Scripts/UI/CombatUI.cs	Add domain power button references
Assets/Scripts/Core/SceneBootstrap.cs	Create domain power buttons
Assets/Scripts/Character/ClassSkillDefinitions.cs	OR hook domain skills in <code>CharacterStats</code>
Assets/Scripts/UI/StatusEffectIndicator.cs	Add icons for active domain powers

Quick-Win Opportunities

1. **CL bonuses (Tier 1)** — Highest impact per line of code. 6 domains get mechanical effect from ~50 lines of core logic + spell tagging.
2. **War domain feat grant** — Simple: 10 lines in `ApplyDomainGrantedPowers()`, immediate mechanical impact (Weapon Focus).
3. **Trickery class skills** — Simple: 5 lines, Bluff + Hide as class skills for Trickery clerics.
4. **Destruction Smite** — Template already exists in `TemplateSmiteSystem`, mostly copy-adapt.

Notes and Caveats

- **Skill list limitations:** The prototype has only 19 skills. Knowledge (Nature), Knowledge (Religion), Disguise, Survival, and Concentration are not in the list. Domain class skill additions for Animal, Knowledge, Plant, and Travel are partially blocked by this.
- **Scroll/Wand system:** Magic domain depends on how much the magic item usage system is fleshed out. Low priority.
- **Speak with Animals (Animal domain):** The spell-like ability is primarily an out-of-combat utility. In a combat prototype, this has minimal mechanical impact. Could be implemented as a simple “you can communicate with Animal creatures” flag.
- **Rebuke vs Turn mechanics:** Rebuking makes creatures cower (10 rounds) and allows commanding (if $HD \leq CL$). This is a different behavior from turning (flee). The TurnUndeadSystem currently only implements “Turn” (flee) and “Destroy” modes.
- **Daily reset:** Domain power daily uses need to reset alongside other daily-use abilities. Find where `TurnUndeadAttemptsUsedToday` is reset and add domain power resets there.